

Peer-to-Peer Networking II — Sync-v2

How a node that is behind catches up to the network — negotiating a sync version, streaming the block backbone in order, then filling in the transaction DAG.

SUBJECT hathor-core · Part II · the node, end to end

CHAPTER 35 · Part II · The Node

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Sync-v2 · SyncVersion · NodeBlockSync · Block streaming · Transaction streaming · Common ancestor · Mempool sync · Backbone-first · Catching up

Table of Contents

Chapter 35 — Peer-to-Peer Networking II: Sync-v2	3
35.1 Localization	3
35.2 What it does and why it exists	5
Why you cannot just ask for everything	5
The generic version of this problem	6
35.3 The concepts it rests on	7
35.4 The code, walked	8
35.4.1 Negotiating a sync version	8
35.4.2 From version to agent: the factory	9
35.4.3 The sync agent: NodeBlockSync	9
35.4.4 Are we even behind? The fast paths	10
35.4.5 Finding the common ancestor	11
35.4.6 Streaming the block backbone	12
35.4.7 Filling in the transaction DAG	13
35.4.8 The mempool sync	15
35.4.9 The message flow, end to end	15
35.5 The design rationale	17
Why backbone-first?	17
Why streaming, not request-per-vertex?	17
35.6 How it plugs into the lifecycle	18
Recap	19

Chapter 35 — Peer-to-Peer Networking II: Sync-v2

What you'll learn in this chapter

- Why a node that has just started cannot "ask for everything" at once — and what dependency order means for a ledger.
- How two peers agree on **which** sync protocol to speak (version negotiation), and why only **sync-v2** survives in the code.
- The **backbone-first** strategy: download the chain of blocks in height order first, then fill in the transaction DAG that each block confirms.
- How sync-v2 finds the **common ancestor** — the boundary between "history I already have" and "history I am missing" — with an n-ary search over block heights.
- The two halves of the protocol: **block streaming** and **transaction streaming**, plus a separate **mempool sync** for unconfirmed transactions.
- How a received vertex re-enters the node through the verification → consensus → ingestion pipeline you met in Chapters 31-33.

This is the canonical deep-dive on synchronization. Chapter 34 covered the plumbing of the peer-to-peer layer: how a node finds peers, opens a connection, performs the handshake, and walks each connection through a state machine until it reaches the **READY** state. This chapter picks up exactly there. Once a connection is **READY**, a sync agent is attached to it, and that agent has one job: make our copy of the ledger match the peer's. Everything below is about how that agent works.

Sync is, frankly, the most stateful and intricate subsystem in the networking layer. We will build it up slowly — first the problem in the abstract, then the concepts it leans on, then the code — and we will hand-trace the message flow so the moving parts settle into place. Take it in order.

35.1 Localization

Sync-v2 lives entirely inside the peer-to-peer package, in its own sub-folder:

```

hathor/p2p/
├── sync_version.py      ← SyncVersion enum (V1_1, V2) + ordering
├── sync_agent.py        ← SyncAgent: abstract interface a sync agent implements
├── sync_factory.py      ← SyncAgentFactory: abstract factory for sync agents
├── manager.py           ← ConnectionsManager: registers/enables sync factories
├── states/
│   ├── hello.py         ← HELLO handshake: versions are negotiated here
│   └── ready.py         ← READY state: sync agent attached & started here
├── sync_v2/
│   └── factory.py        ◀ YOU ARE HERE
│       ├── agent.py      ← SyncV2Factory: a NodeBlockSync per conn
│       ├── streamers.py  ← NodeBlockSync: per-connection driver (brain)
│       ├── blockchain_streaming_client.py ← StreamEnd + the two streaming servers (we→peer)
│       ├── transaction_streaming_client.py ← receives streamed blocks (peer→us)
│       ├── mempool.py    ← receives streamed transactions (peer→us)
│       ├── payloads.py   ← SyncMempoolManager: fetch unconfirmed txs
│       └── exception.py  ← Pydantic models for the wire messages
│                           ← streaming-specific errors

```

The folder splits cleanly into three roles. **agent.py** holds `NodeBlockSync`, the object created once per peer connection that orchestrates the whole catch-up. The **client** files (`blockchain_streaming_client.py`, `transaction_streaming_client.py`) are the receiving side — they consume data the peer pushes at us. The **server** side lives in `streamers.py` — those classes are what we run when we are the peer being asked to send data; another node's client talks to our server. Most nodes do both at once: you are catching up from a peer ahead of you while feeding a peer behind you.

Recap — the P2P connection and the READY state (full treatment in Ch. 34).

Each connection to a peer is driven by a Twisted protocol object that progresses through a state machine: HELLO → PEER-ID → READY. Only when a connection reaches **READY** is it trusted enough to exchange ledger data. The sync agent for a connection is created and started exactly when that connection enters READY — see [hathor/p2p/states/ready.py:127](#) . → full treatment in Ch. 34.

Context. Sync-v2 is what turns a freshly-installed binary into a working full node.

Without it, a node would know only its hard-coded genesis and nothing else; it could never validate a real transaction because it would be missing all the history that transaction depends on. Sync is the bridge between "I just booted with an empty database" and "I hold the same ledger as the rest of the network, and I can independently verify every byte of it."

35.2 What it does and why it exists

Start with the situation. You download `hathor-core`, point it at a data directory, and run `run_node`. Your database contains essentially one thing: the **genesis**¹ — a hard-coded first block and a couple of initial transactions that every Hathor node agrees on by definition. Meanwhile the live network has years of history: millions of blocks and transactions, all of which the peers you are about to connect to already hold.

Sync is the process of closing that gap. It must download every block and every transaction you are missing, and — this is the part that makes it interesting — it must do so in an order that lets you **verify** each one as it arrives.

Why you cannot just ask for everything

The naive idea is: "ask the peer to dump its entire database at me, I'll write it all to disk, done." This fails for a structural reason that comes straight from how the ledger is built.

A full node never trusts a peer's word that a vertex is valid (recall the full in "full node" from Chapter 0). It re-checks everything itself. But to check a transaction, you need the things it depends on to already be present and validated:

- A transaction **spends inputs** — it consumes outputs created by earlier transactions. You cannot confirm "this input is real and unspent" unless you already hold the transaction that created that output.
- A vertex **points at parents** — every vertex links back to two earlier vertices in the DAG. You cannot place a vertex in the graph until its parents are in the graph.
- A block **builds on a previous block** — it names the block before it. You cannot compute a block's height or its accumulated weight until the chain beneath it exists.

So the vertices must arrive in **dependency order**: every vertex's dependencies land before it does. This is exactly a topological order² of the DAG. Hand a node a transaction whose inputs it has never seen and it has no way to judge whether that transaction is spending real coins or inventing them out of thin air. Order is not an optimization here; it is a correctness requirement.

Recap — the DAG and the block backbone (full treatment in Ch. 8 & 9). Hathor's ledger is a DAG³, not a single chain. Every vertex (a **Block** or a **Transaction**) has two kinds of backward edge: **parents** (links into the DAG for confirmation) and, for transactions, **inputs** (the outputs being spent). Blocks form a special spine through this graph — each block names the previous block, so the blocks alone make a linear chain. That chain is the **backbone**. Ordinary transactions hang off the backbone: each transaction is eventually confirmed by a specific block (its **first_block**). → full treatment in Ch. 8 & 9.

The generic version of this problem

Before we look at Hathor's answer, meet the problem in a setting with no blockchain in it at all, because the shape is older and more general than cryptocurrencies.

Imagine a database that is replicated across several servers. One replica has been offline for a week and now rejoins. It is behind. How does it catch up? Two classic approaches:

1. **Snapshot + log.** The lagging replica grabs a checkpoint (a full copy of the state as of some recent point) and then replays the log of every change that happened since. The log must be applied **in order** — you cannot apply "update row 5" before "insert row 5" exists.
2. **Find the divergence point, then stream forward.** Both replicas agree on how far back their histories are identical (the last common entry), and the leader streams every entry after that point, in order, to the follower.

Hathor's sync is approach 2, adapted to a DAG. The "find the divergence point" step becomes **find the common ancestor block**. The "stream entries in order" step becomes **stream the block backbone in height order, then stream the transactions each block confirms**. The reason ordering matters is the same as in the database case: an entry cannot be applied before the entries it depends on.

The five concerns sync must handle, stated plainly:

1. **Agree on a protocol.** Two peers might support different sync versions; they must pick one both understand.
2. **Find the boundary.** Determine the highest point in history where our ledger and the peer's ledger agree — the common ancestor. Below it, we are identical; above it, we are missing data.
3. **Download in dependency order.** Get the missing vertices such that every dependency arrives before the thing that needs it.
4. **Verify and ingest each one.** Run every received vertex through the same validation → consensus → store pipeline as a freshly-broadcast transaction.

5. **Keep up with the present.** Once caught up to the peer's confirmed history, also fetch the unconfirmed transactions sitting in the peer's mempool, and stay subscribed so new vertices arrive in real time.

The rest of the chapter walks each of these in the code.

35.3 The concepts it rests on

Three ideas from earlier chapters carry the weight of this one. Here they are in the local context, with pointers back.

Recap — the height index (full treatment in Ch. 28). The `height index` maps a block height (an integer: 0, 1, 2, ...) to the hash of the block at that height on the best chain. It is what lets the node answer "what is my block at height 5000?" in one lookup instead of walking the chain. Sync leans on it constantly: to compare "the peer's block at height H" against "my block at height H" when hunting for the common ancestor. In the code you will see `self.tx_storage.indexes.height.get(h)` — that is this index. → full treatment in Ch. 28.

Recap — vertex ingestion (full treatment in Ch. 33). When a vertex arrives, it does not go straight to disk. It flows through the **vertex handler**, which runs three stages: **verify** (Ch. 31 — is it well-formed, signatures valid, no double-spend?), **consensus** (Ch. 32 — does it change which history is canonical? does it void anything?), and **store + index** (Ch. 27-28). Sync never bypasses this. Every block and transaction it downloads is handed to `vertex_handler.on_new_block(...)` or `vertex_handler.on_new_mempool_transaction(...)`, the same entry points a freshly-broadcast vertex uses. → full treatment in Ch. 33.

Recap — the reactor and LoopingCall (full treatment in Ch. 16 & 23). Sync is asynchronous: it sends a request, then yields control back to the Twisted reactor⁴ and waits for the reply to arrive as a network event, rather than blocking. The sync loop is driven by a `LoopingCall` — a Twisted timer that re-invokes a method every N seconds. The `@inlineCallbacks` decorator⁵ lets the agent's methods read like straight-line code (`yield self.run_sync_blocks()`) while actually pausing for I/O. → full treatment in Ch. 16 & 23.

With those in hand, we can read the code.

35.4 The code, walked

35.4.1 Negotiating a sync version

Before any ledger data moves, the two peers must agree on a protocol. Hathor models the available protocols as a small enum, `SyncVersion`, in

`hathor/p2p/sync_version.py:20`:

```
@total_ordering
class SyncVersion(Enum):
    V1_1 = 'v1.1'
    V2 = 'v2'
```

There are two named values, but do not be misled: **only V2 is wired up**. The builder registers exactly one factory and enables exactly one version (`hathor/builder/builder.py:88-90`):

```
p2p_manager.add_sync_factory(SyncVersion.V2, sync_v2_factory)
if sync_v2_support == cls.ENABLED:
    p2p_manager.enable_sync_version(SyncVersion.V2)
```

`V1_1` lingers in the enum as a historical artifact — the old **sync-v1** protocol has been removed from the codebase, and no factory backs `V1_1` any longer. Any comment that mentions sync-v1 as a live alternative is stale; treat V2 as the only protocol that exists.

Why keep an enum with two members and an ordering at all, if only one is used? Because the negotiation mechanism is built to be version-agnostic, so that adding a future protocol is a matter of registering another factory, not rewriting the handshake. The negotiation works like this. During the HELLO handshake, each peer advertises the set of sync versions it has enabled (`hathor/p2p/states/hello.py:113`). When a peer receives the other's HELLO, it intersects the two sets and picks the best common one (`hathor/p2p/states/hello.py:122-130`):

```
common_sync_versions = my_sync_versions & remote_sync_versions
if not common_sync_versions:
    # no compatible sync version to use ... just can't connect to this peer
    protocol.send_error_and_close_connection('no compatible sync version to use')
    return

# choose the best version, sorting is implemented in hathor.p2p.sync_versions.__lt__
protocol.sync_version = max(common_sync_versions)
```

`max(...)` works because `SyncVersion` defines an ordering. `__lt__` (`sync_version.py:51`) compares the values returned by `get_priority()` — `V2` returns 20, `V1_1` returns 11 — so the higher-priority version wins when both are supported. If the two peers share no version, the connection is dropped; that is a normal outcome, not an error. The chosen version is stored on the connection as `protocol.sync_version`, and that single field decides which sync agent gets built later.

35.4.2 From version to agent: the factory

Once a connection reaches the READY state, the chosen version is turned into a concrete agent. The READY state looks up the registered factory for the negotiated version and asks it to create an agent (`hathor/p2p/states/ready.py:124-129`):

```
sync_version = self.protocol.sync_version
sync_factory = connections.get_sync_factory(sync_version)
self.sync_agent: SyncAgent = sync_factory.create_sync_agent(self.protocol, reactor=self.reactor)
self.cmd_map.update(self.sync_agent.get_cmd_dict())
```

This is the **factory pattern**⁷ you met in Chapter 3: `SyncV2Factory.create_sync_agent` (`hathor/p2p/sync_v2/factory.py:44`) returns a fresh `NodeBlockSync`, wired with the settings, the connection's protocol, the reactor, the vertex parser, and the vertex handler. One agent per connection. Two lines later, `get_cmd_dict()` registers the agent's message handlers into the connection's command map — that is how an incoming `BLOCKS` or `TRANSACTION` message gets routed to the right method. The agent is then started (`ready.py:144`), which kicks off its main loop.

35.4.3 The sync agent: `NodeBlockSync`

`NodeBlockSync` (`hathor/p2p/sync_v2/agent.py:86`) is the brain. Its docstring is honest about its strategy — "An algorithm to sync two peers based on their blockchain." Blocks first; that is the central design decision, and we will return to why in §35.5.

Its constructor sets up a lot of state (`agent.py:91`), but the fields worth holding in your head are few:

```
self.state = PeerState.UNKNOWN          # where we are in the sync state machine
self.synced_block: Optional[_HeightInfo] = None # highest block we're synced at, w/ this peer
self.peer_best_block: Optional[_HeightInfo] = None # the peer's tip block
self.blk_streaming_client = None         # receives streamed blocks
self.tx_streaming_client = None          # receives streamed transactions
self.mempool_manager = SyncMempoolManager(self) # fetches unconfirmed txs
self._lc_run = LoopingCall(self.run_sync)    # the repeating sync loop
```

`PeerState` (`agent.py:78`) is the sync state machine — distinct from the connection state machine of Chapter 34. It has five members: `UNKNOWN`, `SYNCING_BLOCKS`, `SYNCING_TRANSACTIONS`, `SYNCING_MEMPOOL`, and `ERROR`. They name the phase the agent is in, and reading them in a log tells you exactly where a slow sync is stuck.

`_HeightInfo` (`agent.py:61`) is a tiny `NamedTuple` of `(height, id)` — a block's height paired with its hash. Almost all of the block-syncing logic is expressed in terms of `_HeightInfo`, because comparing "where am I" to "where is the peer" is fundamentally a comparison of (height, hash) pairs.

The agent's main loop is a `LoopingCall` that fires `run_sync` once per second (`agent.py:166`, interval at `agent.py:58`). Each tick is a step of the algorithm — it checks where we are and does the next bit of work. The actual logic is in `_run_sync` (`agent.py:330`):

```
@inlineCallbacks
def _run_sync(self) -> Generator[Any, Any, None]:
    is_block_synced = yield self.run_sync_blocks()
    if is_block_synced:
        # our blocks are synced, so sync the mempool
        yield self.run_sync_mempool()
```

Two phases, in strict order: **first sync the blocks** (the backbone, plus the transactions those blocks confirm), and **only when the blocks are fully caught up**, sync the mempool (the unconfirmed transactions). The mempool is never touched until the confirmed history is complete, because an unconfirmed transaction depends on confirmed ones — order again.

35.4.4 Are we even behind? The fast paths

`run_sync_blocks` (`agent.py:356`) opens by figuring out whether there is anything to do at all. It compares our best block to the peer's:

```
my_best_block = self.get_my_best_block()
self.peer_best_block = yield self.get_peer_best_block()

# Are we synced?
if self.peer_best_block == my_best_block:
    # Yes, we are synced! \o/
    self.update_synced(True)
    self.send_relay(enable=True)
    self.synced_block = self.peer_best_block
    return True
```

`get_peer_best_block` (`agent.py:820`) sends a `GET-BEST-BLOCK` message and waits (via a Twisted `Deferred` ⁶) for the peer to reply with `BEST-BLOCK`, carrying its tip block's hash and height. If the peer's tip equals ours, we are done — same best block, nothing to download. The agent flips to "synced," enables **relay** (asks the peer to forward new vertices in real time from now on), and returns `True`.

There is a second fast path (`agent.py:382`): if the peer's tip is at a height **less than or equal to** ours, and our block at the peer's height is exactly the peer's tip, then the peer is behind us on the same chain — again nothing to fetch from them. This is where the height index earns its keep:

```
common_block_hash = self.tx_storage.indexes.height.get(self.peer_best_block.height)
if common_block_hash == self.peer_best_block.id:
    # nothing to sync because peer is behind me at the same best blockchain
    return True
```

Only if neither fast path fires do we have real work: the peer holds blocks we lack. Now the hard part begins.

35.4.5 Finding the common ancestor

We know the peer is ahead, but we do **not** yet know where our histories diverge. We might share the first 4,000 blocks and differ after that; or a reorg⁸ might mean we agree only up to block 3,500. Downloading must start at the highest block we both have — the **common ancestor**. Streaming from anywhere lower wastes bandwidth re-downloading what we already hold; streaming from anywhere higher leaves a gap and the new blocks won't connect.

A linear scan ("do we share block 1? block 2? block 3? ...") would cost one round-trip per block — far too slow over a network. Instead `find_best_common_block` (`agent.py:557`) runs an **n-ary search** — a generalization of binary search that probes ten heights per round instead of one:

```
hi = min(peer_best_block, my_best_block, key=lambda x: x.height)
lo = _HeightInfo(height=0, id=self._settings.GENESIS_BLOCK_HASH)

while hi.height - lo.height > 1:
    step = math.ceil((hi.height - lo.height) / 10)
    heights = list(range(lo.height, hi.height, step))
    heights.append(hi.height)
    block_info_list = yield self.get_peer_block_hashes(heights)
    ...
```

Read the invariant carefully, because it is the heart of the method:

- `lo` is a height where we are **known to be synced** (it starts at genesis, height 0 — by definition both peers share genesis).
- `hi` is a height where the sync state is **unknown** (it starts at the lower of the two tips — no point searching above where the shorter chain ends).

Each round divides the gap `[lo, hi)` into ten slices and asks the peer, in one message (`GET-PEER-BLOCK-HASHES`, `agent.py:630`), for its block hashes at all eleven boundary heights. The peer replies with `(height, hash)` pairs for the blocks on its best chain at those heights. We then walk that list from highest to lowest and ask, for each: do I have this exact block?

```
for info in block_info_list:
    try:
        blk = self.tx_storage.get_transaction(info.id)
    except TransactionDoesNotExist:
        hi = info          # I don't have it → divergence is at or below here
    else:
        ...                # I do have it → I'm synced at least this high
        lo = info
        break
```

If we have the peer's block at some probed height, then we are synced at least that high — pull `lo` up to it. If we don't, the divergence is at or below that height — pull `hi` down to it. Each round shrinks the unknown gap by a factor of ten, so the search

converges in $\log_{10}(N)$ round-trips — for a chain of a million blocks, about six messages. When `hi - lo` reaches 1, `lo` is the highest block both peers share: the common ancestor. The method returns it.

One safety check is worth noting (`agent.py:589`): the peer's reply must include `lo` itself, because we believe we are synced there. If it does not, a reorg probably happened during the search and the ground shifted under us; the method returns `None` and the loop retries on the next tick. This is the kind of defensive handling that makes sync verbose but resilient — the network is a moving target.

35.4.6 Streaming the block backbone

With the common ancestor in hand, the agent asks the peer to stream every block from the ancestor up to the peer's tip, in order (`agent.py:531`):

```
def start_blockchain_streaming(self, start_block, end_block) -> Deferred[StreamEnd]:
    self._blk_streaming_client = BlockchainStreamingClient(self, start_block, end_block)
    quantity = self._blk_streaming_client._blk_max_quantity
    self.send_get_next_blocks(start_block.id, end_block.id, quantity)
    return self._blk_streaming_client.wait()
```

`send_get_next_blocks` (`agent.py:666`) sends a single `GET-NEXT-BLOCKS` message naming the start hash, the end hash, and how many blocks to send. The peer does **not** reply with one giant message. Instead it streams: it sends a sequence of `BLOCKS` messages, one block each, followed by a final `BLOCKS-END` marker. This is the protocol's defining choice — streaming, not request-per-block — and we will weigh it in §35.5.

On the peer's side, the streaming is run by `BlockchainStreamingServer` (`streamers.py:152`). It walks the best chain forward from the start block, pushing one block per reactor tick via `send_next` (`streamers.py:165`), advancing with `get_next_block_best_chain()` until it reaches the end hash, hits the streaming limit (`DEFAULT_STREAMING_LIMIT = 1000` , `streamers.py:33`), or finds that the chain it is sending has become voided⁹ mid-stream — each outcome is a distinct `StreamEnd` code (`streamers.py:36`). The server is a Twisted push producer, which means Twisted automatically pauses it if our connection's send buffer fills up and resumes it when the buffer drains — flow control for free.

On our side, each `BLOCKS` message lands in `handle_blocks` (`agent.py:762`), which deserializes the bytes back into a `Block` and hands it to the receiving client, `BlockchainStreamingClient.handle_blocks` (`blockchain_streaming_client.py:77`). That method does three things per block:

1. **Sanity-checks the count and linearity.** If more blocks arrive than were requested, or a block does not connect to the previously received one (`blk.get_block_parent_hash() != last_block.hash`), the stream fails (`blockchain_streaming_client.py:114-122`). This catches a misbehaving or buggy peer.

2. **Detects when we've caught up.** If a streamed block is one we already have and isn't voided, we've reached known territory; after enough repeats it stops the stream early (`blockchain_streaming_client.py:100-112`). No point re-downloading.
3. **Either ingests the block or sets it aside.** Here is the subtle part:

```
if self.tx_storage.can_validate_full(blk):
    self.vertex_handler.on_new_block(blk, deps=[])
else:
    self._partial_blocks.append(blk)
```

A block can be fully validated only if every transaction it depends on is already present. During a fresh sync that is usually **not** the case — the block confirms transactions we have not downloaded yet. So the client checks `can_validate_full`: if the block's dependencies are all here, ingest it immediately (`on_new_block` with an empty `deps` list). If not, park it in `_partial_blocks` — a list of blocks that are downloaded but cannot yet be fully validated, because their transactions are still missing. Those parked blocks drive the next phase.

35.4.7 Filling in the transaction DAG

Back in `run_sync_blocks` (`agent.py:422`), once the block stream finishes, the agent checks for parked blocks:

```
partial_blocks = self._blk_streaming_client._partial_blocks
if partial_blocks:
    self.state = PeerState.SYNCING_TRANSACTIONS
    reason = yield self.start_transactions_streaming(partial_blocks)
    while reason == StreamEnd.LIMIT_EXCEEDED:
        reason = yield self.resume_transactions_streaming()
```

Now the agent streams the transactions those blocks confirm.

`start_transactions_streaming` (`agent.py:858`) sends a `GET-TRANSACTIONS-BFS` message naming the first and last parked block, and a receiving client, `TransactionStreamingClient` (`transaction_streaming_client.py:42`), is created to consume the reply. The `while` loop handles the case where there are more transactions than one stream's limit allows: if the stream ends with `LIMIT_EXCEEDED`, it resumes from where it left off (`agent.py:874`) until everything is delivered.

The peer answers from `TransactionsStreamingServer` (`streamers.py:207`). Its docstring captures the strategy: "Streams all transactions confirmed by the given block, from right to left (decreasing timestamp)." For each parked block in turn, it runs a **breadth-first walk** of the DAG (`BFSOrderWalk`, `streamers.py:235`) starting from that block and walking backward through parents and inputs, emitting every transaction the block confirms. It skips transactions that belong to an earlier block (those were confirmed

before this block and will be delivered with that earlier block instead — `streamers.py:300`), so each transaction is sent exactly once, attributed to the first block that confirms it.

Why breadth-first, and why the `start_from` mechanism? Two reasons. First, BFS over the confirmation graph naturally produces transactions in an order close to dependency order. Second, the `GET-TRANSACTIONS-BFS` message carries a `start_from` list (`payloads.py:37`) so that a resumed stream can pick up from a precise frontier rather than restarting the whole walk — that is what makes the `LIMIT_EXCEEDED` resume loop above cheap.

On our side, the receiving client is careful about ordering, because BFS does not perfectly guarantee that a transaction's dependencies arrive before it does. The client maintains a small staging area (`transaction_streaming_client.py`):

- `_waiting_for` — the set of dependency hashes it still expects to receive.
- `_db` — transactions received but held back, waiting for their dependencies.

Each incoming `TRANSACTION` lands in `handle_transaction` (`agent.py:1016` → `transaction_streaming_client.py:123`), is queued, and processed one at a time by `process_queue` (`transaction_streaming_client.py:143`). For each transaction, `_process_transaction` (`transaction_streaming_client.py:166`):

1. Runs **basic verification** (`verify_basic`, `transaction_streaming_client.py:173`) — a cheap, dependency-free sanity check (structure, weight) before doing anything expensive. (Full validation comes later, at ingestion.)
2. Updates the dependency bookkeeping: looks at everything this transaction needs (`get_all_dependencies`) and sorts each into "already have it" or "still waiting" (`_update_dependencies`, `transaction_streaming_client.py:212`).
3. When `_waiting_for` becomes empty — every dependency for the current block is now in hand — it flushes the staged transactions and the block together, in timestamp order, by calling `_execute_and_prepare_next` (`transaction_streaming_client.py:239`):

```
blk = self.partial_blocks[self._idx]
vertex_list = list(self._db.values())
vertex_list.sort(key=lambda v: v.timestamp)
yield self.sync_agent.on_block_complete(blk, vertex_list)
```

`on_block_complete` (`agent.py:612`) is the payoff. It hands the block **and** its full set of confirmed transactions to the vertex handler at once:

```
yield self.vertex_handler.on_new_block(blk, deps=vertex_list)
```

Now the block can be fully validated, because `deps=vertex_list` carries exactly the transactions it was waiting on. The vertex handler runs them through verification → consensus → store (Chapter 33). The client then advances to the next parked block and repeats. Block by block, the backbone and its transaction DAG are committed to our ledger in dependency order.

35.4.8 The mempool sync

After the confirmed history is fully caught up, `_run_sync` proceeds to the mempool (`agent.py:343`). The **mempool**¹⁰ is the set of valid transactions a peer knows about that have been seen but not yet confirmed by any block. They are not part of the backbone — no block points at them yet — so they need their own fetch.

`SyncMempoolManager` (`mempool.py:30`) handles this. It first asks the peer for its mempool **tips** (`GET-TIPS`, the transactions at the frontier of the unconfirmed DAG), keeping only those we don't already have (`mempool.py:90-93`). Then, for each missing tip, it runs a **depth-first search** down the dependency graph (`_dfs`, `mempool.py:108`):

```
def _next_missing_dep(self, tx):
    for txin in tx.inputs:
        if not self.tx_storage.transaction_exists(txin.tx_id):
            return txin.tx_id
    for parent in tx.parents:
        if not self.tx_storage.transaction_exists(parent):
            return parent
    return None
```

For a given transaction, it finds the first dependency (an input or a parent) it does not yet have, downloads that first, and only adds the transaction itself once all its dependencies are present (`_add_tx`, `mempool.py:137`). This is dependency order enforced explicitly, one transaction at a time, via the DFS stack — the unconfirmed counterpart of what block streaming did for confirmed history. Each fully-resolved transaction is handed to `vertex_handler.on_new_mempool_transaction` (`mempool.py:143`) and, if accepted, re-broadcast to our other peers.

35.4.9 The message flow, end to end

Here is one full catch-up against a peer that is ahead of us, as a sequence diagram. Time flows downward; → is a message we send, ← is one we receive.



Once the agent reaches the bottom, it is synced with this peer and has enabled relay. From here, each new block or transaction the peer learns about is pushed to us live, re-entering the same ingestion pipeline. The `LoopingCall` keeps ticking; if either side advances, the agent steps through the relevant phases again. Sync is never truly "finished" — it is a standing relationship that keeps two ledgers aligned.

35.5 The design rationale

Two decisions define sync-v2. Each is a trade-off worth understanding, because the alternatives are not obviously wrong.

Why backbone-first?

The agent downloads **blocks before transactions**, always. Why not just fetch transactions in topological order directly and let the blocks fall out?

Because the blocks give the download a **spine to hang everything on**. The block chain is linear — each block names exactly one previous block — so "download the blocks" has an unambiguous order: by height, low to high. There is no ambiguity about what comes next. The transaction DAG, by contrast, is a tangle with no single linear order; trying to stream it directly, with no anchor, would mean constantly answering "where am I, and what should come next?" with graph traversals over an enormous structure.

By fetching the backbone first, sync converts a hard problem (order a giant DAG) into two easy ones: (1) order a linear chain — trivial, it's just heights; and (2) for each block, fetch the small bag of transactions that block confirms — a bounded, local problem. Each block acts as a checkpoint that says "every transaction confirmed at or below me is now accounted for." The block backbone is, in effect, a pre-built table of contents for the DAG. That is the deep reason this chapter's subtitle calls it "streaming the block backbone in order, then filling in the transaction DAG."

The common-ancestor search exists for the same reason: it finds where on the backbone to start, so we re-download nothing and leave no gaps.

Why streaming, not request-per-vertex?

Sync could have been built as a simple loop: "request block N, wait, receive it, request block N+1...". That is easier to reason about. Sync-v2 instead **streams**: one `GET-NEXT-BLOCKS` triggers a flood of `BLOCKS` messages with a single `BLOCKS-END` at the close.

The reason is **throughput**. A request-per-vertex design pays one full network round-trip per vertex. Over a link with 50 ms latency, that caps you at ~20 vertices per second no matter how fat the pipe — you spend almost all your time waiting for the next request to travel there and the reply to travel back. Streaming decouples sending from acknowledging: the peer keeps blocks flowing continuously, limited only by bandwidth, not by round-trip time. For catching up across millions of vertices, that difference is the difference between minutes and days.

Streaming brings its own costs, and the code pays them: flow control (the Twisted push-producer machinery in `streamers.py`, so a fast sender doesn't overrun a slow receiver), bounded buffers (the `DEFAULT_STREAMING_LIMIT` and the resume loop), the staging area in the transaction client (because streamed order isn't a perfect

dependency order), and a watchdog that kills a stream that has gone stale (`agent.py:296`). This is why sync is the most complex stateful subsystem in the networking layer — and it is honest to say so. The complexity buys throughput, and throughput is what makes a fresh node's catch-up finish in a reasonable time.

35.6 How it plugs into the lifecycle

Sync does not run in isolation; it sits at the meeting point of half the node. Tracing the connections, with chapter pointers:

- **It is attached at READY (Ch. 34).** A `NodeBlockSync` is created exactly when a peer connection reaches the READY state (`ready.py:127`) and started immediately (`ready.py:144`). One agent per connection; the node runs several at once, one per peer.
- **It runs on the reactor (Ch. 16 & 23).** The whole algorithm is timer-driven (`LoopingCall` , `agent.py:166`) and non-blocking (`@inlineCallbacks` + `Deferred` throughout). It never stalls the node; between messages, the reactor is free to serve every other peer and client.
- **Every received vertex goes through ingestion (Ch. 31-33).** Sync never writes to storage directly. A downloaded block reaches the ledger through `vertex_handler.on_new_block` (`agent.py:617` , `blockchain_streaming_client.py:131`); a downloaded mempool transaction through `vertex_handler.on_new_mempool_transaction` (`mempool.py:143`). That handler runs verification (Ch. 31) and consensus (Ch. 32) on each one. Sync's job is delivery in the right order; judging validity stays with the same pipeline that judges a freshly-broadcast transaction.
- **It leans on the height index (Ch. 28).** The fast-path check and the common-ancestor search both query `tx_storage.indexes.height.get(...)` to map a height to a best-chain block hash.
- **It serializes vertices with the wire codec (Ch. 26).** Streamed blocks and transactions cross the wire as the bespoke binary format; `handle_blocks` and `handle_transaction` call `vertex_parser.deserialize(...)` (`agent.py:773` , `agent.py:1023`) to turn bytes back into `Block` and `Transaction` objects.

Put differently: sync is the input pump for the entire ingestion machine. Chapter 33 told you what happens to a vertex once it is inside the node; this chapter told you how the vertices that aren't created locally get into the node in the first place — and in an order the rest of the machine can actually use.

Recap

Concern (§35.2)	Mechanism	Where in the code
Agree on a protocol	HELLO advertises versions; <code>max(common)</code> picks the best; only V2 is registered	<code>hello.py:122-130</code> , <code>builder.py:88-90</code>
Build the agent	Factory creates one <code>NodeBlockSync</code> per READY connection	<code>factory.py:44</code> , <code>ready.py:127</code>
Drive the algorithm	A 1-second <code>LoopingCall</code> steps the state machine; blocks then mempool	<code>agent.py:166</code> , <code>_run_sync</code> <code>agent.py:330</code>
Find the boundary	n-ary search over block heights → common ancestor	<code>find_best_common_block</code> <code>agent.py:557</code>
Stream the backbone	<code>GET-NEXT-BLOCKS</code> → streamed <code>BLOCKS</code> → <code>BLOCKS-END</code> , in height order	<code>streamers.py:152</code> , <code>blockchain_streaming_client.py:77</code>
Fill in the DAG	<code>GET-TRANSACTIONS-BFS</code> ; stage txs until deps complete, then ingest block+txs	<code>transaction_streaming_client.py:166</code> , <code>agent.py:612</code>
Sync the mempool	<code>GET-TIPS</code> then DFS each tip down its missing dependencies	<code>mempool.py:30</code> , <code>_dfs</code> <code>mempool.py:108</code>
Ingest each vertex	Hand to the vertex handler: verify → consensus → store	<code>on_new_block</code> / <code>on_new_mempool_transaction</code> (Ch. 33)

Sync-v2 is the bridge from "I booted with nothing" to "I hold and independently verify the whole ledger." Its one structural idea — **backbone first, DAG second** — turns the impossible-looking task of ordering a giant graph into the tractable one of walking a linear chain and, for each link, fetching the small set of transactions it confirms. Its one performance idea — **stream, don't poll** — is what makes that catch-up finish in minutes rather than days, at the cost of the stateful machinery that fills `agent.py` . Everything it downloads re-enters the node through the same verification and consensus pipeline as any other vertex, so a synced node is no more trusting of its peers than a node that mined every block itself.

The next chapter turns to a different network surface: `hathor/websocket`. Where this chapter was about peers exchanging ledger data, Chapter 36 is about **clients** — wallets and dashboards — subscribing to a live feed of events from the node. Same reactor, same vertices, but a one-way stream out to consumers rather than a two-way sync between equals.

1. The genesis is the hard-coded starting point of the ledger — the first block and initial transactions — that every node agrees on by definition. A fresh node holds only genesis until sync fills in the rest. ↩
2. A topological order of a directed acyclic graph is a linear ordering of its vertices such that every vertex comes after all the vertices it depends on (points to). Processing a DAG in topological order guarantees you never reach an item before its prerequisites. Full treatment of the DAG in Ch. 8. ↩
3. **DAG** = Directed Acyclic Graph: items ("vertices") joined by one-way links ("edges") with no way to follow the links in a loop. Hathor's ledger is a DAG of blocks and transactions. Ch. 8. ↩
4. The reactor is Twisted's central event loop: it waits for events (network data, timers) and calls the right piece of code in response, so the program never blocks on I/O. Ch. 16 & 23. ↩
5. `@inlineCallbacks` is a Twisted decorator that lets you write asynchronous code as if it were straight-line: `yield some_deferred` pauses the function until the result arrives, instead of nesting callbacks. Ch. 16. ↩
6. A Deferred is Twisted's name for a future — a placeholder for a value that will exist later (e.g. a peer's reply). You attach code to run when it resolves, rather than blocking to wait. Ch. 16. ↩
7. The factory pattern is a design pattern where a dedicated object's job is to construct other objects, hiding the details of which concrete class is built. `SyncV2Factory` builds a `NodeBlockSync` per connection. Ch. 3. ↩
8. A reorg (reorganization) happens when the network's view of the best chain changes — a competing chain becomes heavier and replaces part of the previously-best one. During sync, a reorg can shift the common ancestor mid-search. Ch. 10 & 32. ↩
9. A vertex is voided when consensus marks it as not part of canonical history (e.g. it was on a chain that lost a reorg, or it conflicts with a heavier transaction). Voided does not mean deleted — it means "recorded but not counted." Ch. 10 & 32. ↩
10. The mempool ("memory pool") is the set of valid transactions a node knows about that have been seen but not yet confirmed by any block. Ch. 28. ↩